

Consider the following Python dictionary data and Python list labels:

```
data = {'birds': ['Cranes', 'Cranes', 'plovers', 'spoonbills', 'spoonbills', 'Cranes', 'plovers', 'Cranes',  
'spoonbills', 'spoonbills'], 'age': [3.5, 4, 1.5, np.nan, 6, 3, 5.5, np.nan, 8, 4], 'visits': [2, 4, 3, 4, 3, 4,  
2, 2, 3, 2], 'priority': ['yes', 'yes', 'no', 'yes', 'no', 'no', 'no', 'yes', 'no', 'no']}
```

```
labels = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j']
```

1. Create a DataFrame birds from this dictionary data which has the index labels.

```
import pandas as pd  
import numpy as np  
  
data = {  
    'birds': ['Cranes', 'Cranes', 'plovers', 'spoonbills',  
'spoonbills', 'Cranes', 'plovers', 'Cranes', 'spoonbills',  
'spoonbills'],  
    'age': [3.5, 4, 1.5, np.nan, 6, 3, 5.5, np.nan, 8, 4],  
    'visits': [2, 4, 3, 4, 3, 4, 2, 2, 3, 2],  
    'priority': ['yes', 'yes', 'no', 'yes', 'no', 'no', 'no', 'yes', 'no', 'no']  
}  
  
labels = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j']  
  
birds = pd.DataFrame(data, index=labels)  
  
print(birds)
```

	birds	age	visits	priority
a	Cranes	3.5	2	yes
b	Cranes	4.0	4	yes
c	plovers	1.5	3	no
d	spoonbills	NaN	4	yes
e	spoonbills	6.0	3	no
f	Cranes	3.0	4	no
g	plovers	5.5	2	no
h	Cranes	NaN	2	yes
i	spoonbills	8.0	3	no
j	spoonbills	4.0	2	no

2. Display a summary of the basic information about birds DataFrame and its data.

```
birds.info()  
  
<class 'pandas.core.frame.DataFrame'>  
Index: 10 entries, a to j  
Data columns (total 4 columns):  
#   Column      Non-Null Count  Dtype  
---  -  
a   birds      10 non-null    object  
b   age         9 non-null     float64  
c   visits      10 non-null    int64  
d   priority    10 non-null    object
```

```

0    birds      10 non-null    object
1    age        8 non-null    float64
2    visits     10 non-null    int64
3    priority   10 non-null    object
dtypes: float64(1), int64(1), object(2)
memory usage: 400.0+ bytes

```

```
birds.describe()
```

```

      age  visits
count  8.000000  10.000000
mean   4.437500  2.900000
std    2.007797  0.875595
min    1.500000  2.000000
25%    3.375000  2.000000
50%    4.000000  3.000000
75%    5.625000  3.750000
max    8.000000  4.000000

```

```
birds.head()
```

```

   birds  age  visits  priority
a  Cranes  3.5      2      yes
b  Cranes  4.0      4      yes
c  plovers  1.5      3       no
d  spoonbills  NaN      4      yes
e  spoonbills  6.0      3       no

```

```
birds.isnull().sum()
```

```

birds      0
age        2
visits     0
priority    0
dtype: int64

```

3. Print the first 2 rows of the birds dataframe

```
birds.head(2)
```

```

   birds  age  visits  priority
a  Cranes  3.5      2      yes
b  Cranes  4.0      4      yes

```

4. Print all the rows with only 'birds' and 'age' columns from the dataframe

```
birds[['birds', 'age']]
```

```

   birds  age
a  Cranes  3.5

```

b	Cranes	4.0
c	plovers	1.5
d	spoonbills	NaN
e	spoonbills	6.0
f	Cranes	3.0
g	plovers	5.5
h	Cranes	NaN
i	spoonbills	8.0
j	spoonbills	4.0

5. select [2, 3, 7] rows and in columns ['birds', 'age', 'visits']

```
birds.iloc[[2, 3, 7]][['birds', 'age', 'visits']]
```

	birds	age	visits
c	plovers	1.5	3
d	spoonbills	NaN	4
h	Cranes	NaN	2

6. select the rows where the number of visits is less than 4

```
birds[birds['visits'] < 4]
```

	birds	age	visits	priority
a	Cranes	3.5	2	yes
c	plovers	1.5	3	no
e	spoonbills	6.0	3	no
g	plovers	5.5	2	no
h	Cranes	NaN	2	yes
i	spoonbills	8.0	3	no
j	spoonbills	4.0	2	no

7. select the rows with columns ['birds', 'visits'] where the age is missing i.e NaN

```
birds.loc[birds['age'].isna(), ['birds', 'visits']]
```

	birds	visits
d	spoonbills	4
h	Cranes	2

8. Select the rows where the birds is a Cranes and the age is less than 4

```
birds[(birds['birds'] == 'Cranes') & (birds['age'] < 4)]
```

	birds	age	visits	priority
a	Cranes	3.5	2	yes
f	Cranes	3.0	4	no

9. Select the rows the age is between 2 and 4(inclusive)

```
birds[birds['age'].between(2,4)]
```

	birds	age	visits	priority
a	Cranes	3.5	2	yes
b	Cranes	4.0	4	yes
f	Cranes	3.0	4	no
j	spoonbills	4.0	2	no

10. Find the total number of visits of the bird Cranes

```
birds[birds['birds'] == 'Cranes']['visits'].sum()
```

```
12
```

11. Calculate the mean age for each different birds in dataframe.

```
birds.groupby('birds')['age'].mean()
```

```
birds
Cranes      3.5
plovers     3.5
spoonbills  6.0
Name: age, dtype: float64
```

12. Append a new row 'k' to dataframe with your choice of values for each column. Then delete that row to return the original DataFrame.

```
birds.loc['k'] = ['Cranes', 5.0, 3, 'yes']
```

```
birds = birds.drop('k')
```

13. Find the number of each type of birds in dataframe (Counts)

```
birds['birds'].value_counts()
```

```
Cranes      4
spoonbills  4
plovers     2
Name: birds, dtype: int64
```

14. Sort dataframe (birds) first by the values in the 'age' in descending order, then by the value in the 'visits' column in ascending order.

```
birds.sort_values(by=['age', 'visits'], ascending=[False, True])
```

	birds	age	visits	priority
i	spoonbills	8.0	3	no
e	spoonbills	6.0	3	no
g	plovers	5.5	2	no

j	spoonbills	4.0	2	no
b	Cranes	4.0	4	yes
a	Cranes	3.5	2	yes
f	Cranes	3.0	4	no
c	plovers	1.5	3	no
h	Cranes	NaN	2	yes
d	spoonbills	NaN	4	yes

15. Replace the priority column values with 'yes' should be 1 and 'no' should be 0

```
birds['priority'] = birds['priority'].map({'yes': 1, 'no': 0})
print(birds)
```

	birds	age	visits	priority
a	trumpeters	3.5	2	NaN
b	trumpeters	4.0	4	NaN
c	plovers	1.5	3	NaN
d	spoonbills	NaN	4	NaN
e	spoonbills	6.0	3	NaN
f	trumpeters	3.0	4	NaN
g	plovers	5.5	2	NaN
h	trumpeters	NaN	2	NaN
i	spoonbills	8.0	3	NaN
j	spoonbills	4.0	2	NaN

16. In the 'birds' column, change the 'Cranes' entries to 'trumpeters'.

```
birds['birds'] = birds['birds'].replace('Cranes', 'trumpeters')
print(birds)
```

	birds	age	visits	priority
a	trumpeters	3.5	2	NaN
b	trumpeters	4.0	4	NaN
c	plovers	1.5	3	NaN
d	spoonbills	NaN	4	NaN
e	spoonbills	6.0	3	NaN
f	trumpeters	3.0	4	NaN
g	plovers	5.5	2	NaN
h	trumpeters	NaN	2	NaN
i	spoonbills	8.0	3	NaN
j	spoonbills	4.0	2	NaN